We haven't really talked about loops yet, but each takes a block argument and executes the block once for each element in the array, in this case, checking the hash for each word and incrementing the count associated with that word.

One little job left. The hash containing the word frequencies is ordered based on the first time it sees each word. It would be better to display the results based on the frequencies of the words. We can do that using the hash's sort_by method. When you use sort_by, you give it a block that tells the sort what to use when making comparisons. In our case, we'll just use the count. The result of the sort is an array containing a set of two-element arrays, with each subarray corresponding to a key/entry pair in the original hash. This makes our whole program:

tut_containers/word_freq/ugly_word_count.rb

```
require_relative "words_from_string"
require_relative "count_frequency"

raw_text = "The problem breaks down into two parts. First, given some text
as a string, return a list of words. That sounds like an array. Then, build
a count for each distinct word. That sounds like a use for a hash---we can
index it with the word and use the corresponding entry to keep a count."

word_list = words_from_string(raw_text)
counts = count_frequency(word_list)
sorted = counts.sort_by { |word, count| count }
top_five = sorted.last(5)

top_five.reverse_each do |word, count|
  puts "#{word}:  #{count}"
end
```

produces:

```
a:  6
the:  3
count:  2
word:  2
sounds:  2
```

Note that the sorted array is low to high, so we use last to take the last five elements of the array, meaning the one with the highest count, and then reverse_each to iterate them highest to lowest.

At this point, a quick test may be in order to validate our code. These tests are going to be valuable in a moment because we're going to change that code into more commonly used Ruby and we want to make sure the behavior doesn't change.

To do this, we're going to use a testing framework called Minitest that comes with the standard Ruby distributions. We won't describe it fully yet (we do that in Chapter 13, Testing Ruby Code, on page 207). For now, we'll just say that the class MiniTest::Test brings in testing functionality, including the method assert_equal, which checks that its two parameters are equal, complaining bitterly if they aren't. We'll use assertions to test our two methods, one method at a time. (That's one reason why we wrote them as separate methods—it makes them testable in isolation.)

Here are some tests for the word_from_string method:

tut_containers/word_freq/test_words_from_string.rb

```
require_relative "words_from_string"
```